

Reliable Program Reasoning through Learning and Formal Feedback

Jian Wang, PhD · Research statement for Program Reasoning · September 2026

I study how learning-based models can reason about programs while remaining accountable to executable and formal evidence. Code models can propose plausible invariants, patches, and explanations, but plausibility is not proof: an invariant must be inductive, a patch must satisfy tests and specifications, and a result must survive changes in program family and toolchain. My aim is to couple learned proposal mechanisms with semantic checks, counterexamples, and benchmarks that expose failure rather than hide it.

This agenda fits the Program Reasoning project at Imperial Global Singapore because it sits directly between artificial intelligence, programming languages, and formal methods. I would pursue three connected questions: what semantic abstractions a model should learn, how formal feedback should change the model, and how to evaluate whether the resulting reasoning is robust.

Research direction

1. Learn abstractions that formal tools can check. Loop invariants, specifications, summaries, and repair conditions compress program behaviour into objects that can be verified. I want to learn these objects from programs while keeping the proof obligation explicit. The initial setting is loop-invariant synthesis: generate candidate programs, obtain candidate invariants, reject inconsistent translations, and train models on reasoning paired with verifier-accepted results. The next step is to move beyond single loops to multiple loops, functions, arrays, and pointer-heavy C/C++ code. I would compare direct generation, supervised fine-tuning, retrieval, and verifier-guided refinement. The main measures would be verifier success, invariant strength, transfer to withheld program families, solver calls, latency, and compute—not textual similarity to a reference invariant.

2. Turn formal feedback into a useful learning signal. A solver can reject a candidate and sometimes provide a counterexample, but the signal is sparse and the interface between source code and the verifier can itself be wrong. I would study structured training signals derived from failed proof obligations, counterexamples, cross-verifier disagreement, and round-trip semantic checks. A generator and checker should remain separate enough that a model cannot receive credit merely for exploiting one encoding. Experiments would compare supervised initialization, reinforcement learning from formal rewards, counterexample-guided fine-tuning, curricula, and verifier ensembles. Success would require improvement on held-out programs without weakening output validity or increasing solver cost beyond a stated budget.

3. Evaluate reasoning under distribution shift. Program-reasoning benchmarks can overestimate progress when train and test programs share templates, when specifications are weak, or when a single verifier defines correctness. I would build evaluation splits by program family and transformation rather than only by random sampling. Tests would include semantics-preserving refactorings, harder control flow, altered data representations, and independently executable outcomes. Every result would report coverage, failure categories, solver calls, wall-clock time, and compute alongside aggregate success. Negative results would be first-class evidence: a method that helps linear arithmetic but fails on arrays should support only the narrower claim.

Research basis

[Loop-R1](#) is my most direct starting point. It builds a semantic data pipeline for loop-invariant synthesis and compares supervised fine-tuning, reinforcement learning, and their combination across three small code models. The pipeline reduces 1.53 million generated candidates to 172,000 valid programs, 58,275 inferred invariants, 42,638 semantically filtered pairs, and 28,420 reasoning triples. On Clause2Inv, Qwen3-1.7B improves from 19.39% pass@10 in the base model to 61.25% after supervised and reinforcement learning; under LaM4Inv, success rises from 37.5% to 77.8%. The paper also sets an important boundary: reinforcement learning alone is weak or unstable, and performance on InvBench remains low. These results motivate better feedback and harder generalization tests; they do not establish a general solution to program reasoning.

My earlier work supplies complementary evidence. [Defects4C](#) provides 248 real buggy functions and 102 vulnerable functions with executable reproduction tests, and evaluates 24 models on C/C++ repair. It gives a realistic substrate for studying whether learned reasoning transfers from invariant benchmarks to repair. Our [execution-trace study](#) found that trace information brought limited gains in the evaluated fine-tuning and test-time settings. That result cautions against assuming that more semantic tokens automatically yield better reasoning. [RATCHET](#) combines retrieval, learned fault localisation, and patch generation; it provides a baseline for comparing retrieved examples with verifier-produced feedback.

Across these projects, my contribution is empirical neuro-symbolic program reasoning: constructing executable datasets, training code models, integrating analysis tools, and designing evaluations with auditable outcomes. My prior work does not by itself demonstrate new theorem-proving results. At Imperial I would deepen the formal side in collaboration with programming-languages and verification researchers, while contributing the model-training, systems, and benchmark expertise needed to test ideas at scale.

Initial programme

In the first stage, I would reproduce and stress-test Loop-R1 under family-level splits, stronger translation checks, and at least two independent verification paths. The output would be a failure taxonomy and a reproducible baseline, not only a higher score. In the second stage, I would train with structured counterexample and proof-obligation feedback, then measure which signals improve unseen-program performance and which merely optimize one verifier. In the third stage, I would connect invariant reasoning to repair on Defects4C, asking whether learned abstractions help localise a fault, constrain a patch, or explain why a candidate fails.

The resulting artifacts should include a benchmark with leakage-resistant splits, an open evaluation harness, trained checkpoints where release is appropriate, and a record of unsuccessful configurations. The scientific question is not whether neural or symbolic methods should replace one another. It is which division of labour makes program reasoning more correct, more general, and easier to audit.

Fit and longer-term goal

Imperial's programme offers the right setting for this work: program understanding, verification, and synthesis are treated as connected problems, while neural and formal methods can be evaluated through concrete tools. I can contribute experience spanning learning systems, program repair, reproducible evaluation, and production software infrastructure. I would work across model, verifier, and benchmark boundaries, and publish the assumptions and costs that make each result interpretable.

My longer-term goal is a program-reasoning system that proposes useful semantic artifacts, learns from precise formal feedback, and knows when its evidence is insufficient. Such a system would not replace verification. It would make formal reasoning easier to apply while leaving correctness claims anchored to explicit, independently checkable obligations.